



Deresegn POS

System & Architecture Design (SDD)

System Design Document

Document Version: 1.0

Application Version: 1.0.3+4

Date: August 5, 2026

Classification: Confidential

Prepared by: Micro Sun & Solution PLC

Email: amanuielt@mssethiopia.com

Phone: +251 911 058 179



Deresegn POS - System & Architecture Design Document (SDD)

1. Mobile App Architecture Overview

1.1 High-Level Description

Deresegn POS is a cross-platform mobile application built with Flutter (Dart) for Android and iOS. The app follows a client-server architecture where the mobile client handles user interaction, local caching, and offline queuing, while all fiscal operations (invoice signing, MoR API communication) are processed through a centralized Laravel 8 (PHP) REST API backend.

1.2 Architecture Pattern

The mobile application implements a layered architecture organized into the following tiers:

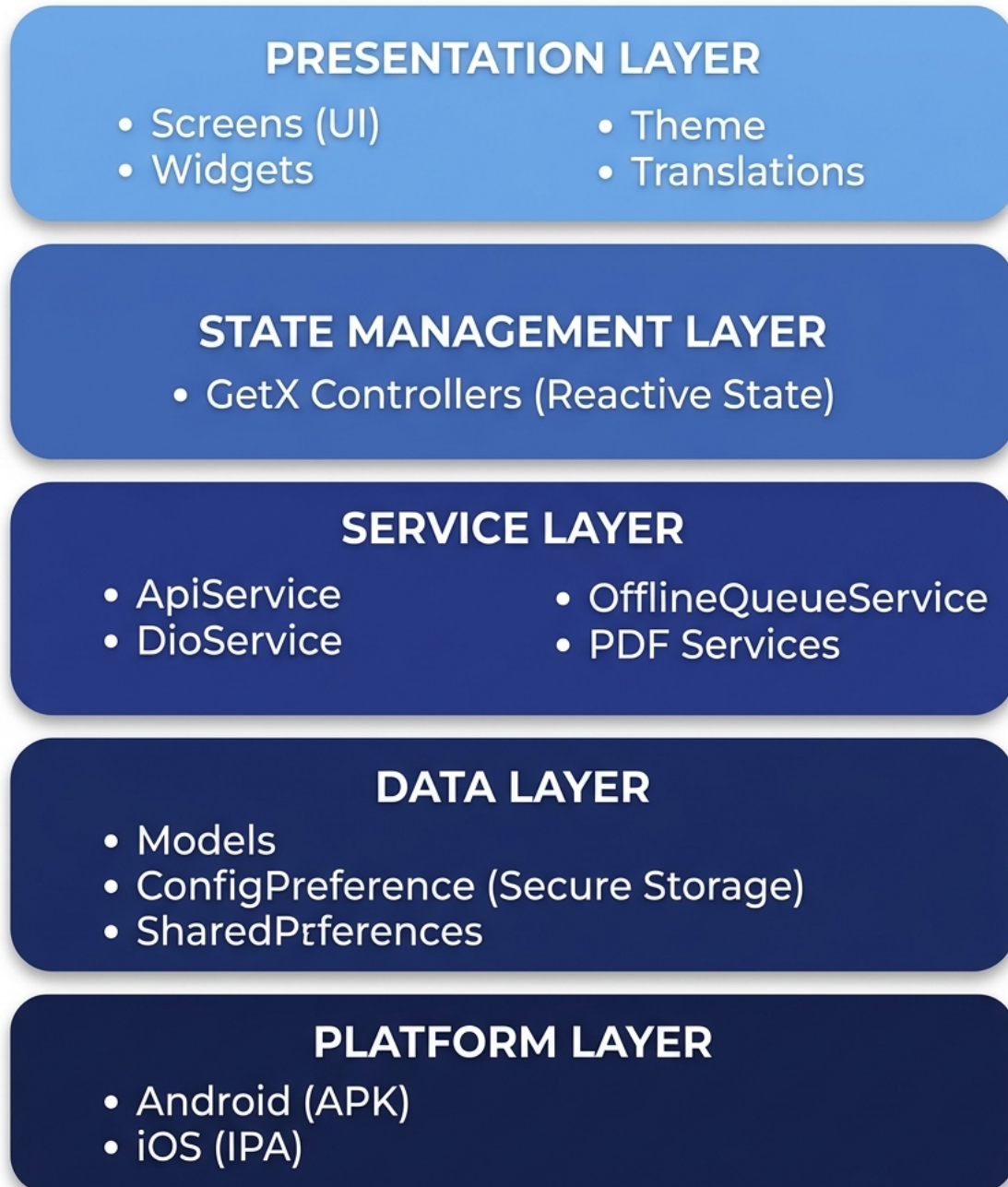


Figure: Mobile Application Layer Architecture

1.3 Backend Architecture

Deresegn POS

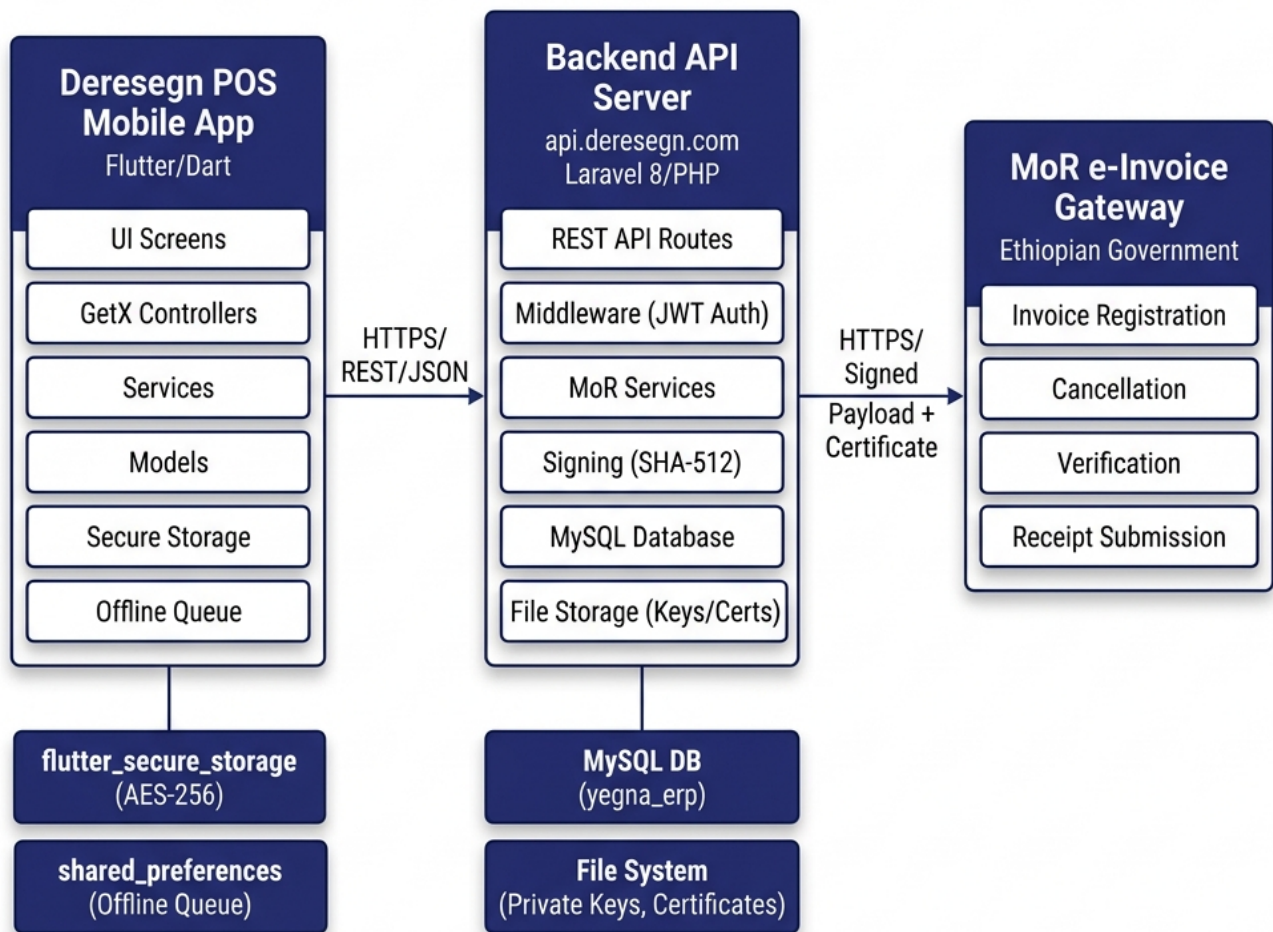


Figure: System Architecture Diagram

2. Technology Stack



2.1 Mobile Application (Client)

Component	Technology	Version
Framework	Flutter	SDK ^3.12.2
Language	Dart	^3.0.0
State Management	GetX	^4.7.3
HTTP Client	Dio	^5.10.0
Secure Storage	flutter_secure_storage	^10.3.1
Local Storage	shared_preferences	^2.5.5
JWT Handling	jwt_decoder	^2.0.1
PDF Generation	pdf + printing	^3.11.2, ^5.13.2
QR Code	qr_flutter	^4.1.0
Connectivity	connectivity_plus	^7.3.1
Cookie Management	cookie_jar	^4.0.9
File Handling	file_picker, path_provider	^10.2.0, ^2.1.6
Localization	intl	^0.20.3
Navigation	persistent_bottom_nav_bar	^6.2.1
Logging	logger	^2.7.0
Typography	Inter, NotoSansEthiopic, JetBrains Mono	Custom bundled fonts

2.2 Backend API (Server)

Component	Technology	Version
Framework	Laravel	^8.75
Language	PHP	^7.3 / ^8.0
Authentication	tymon/jwt-auth	^1.0
API Auth	Laravel Sanctum	^2.11
Database	MySQL	5.7+
HTTP Client	Guzzle	^7.0.1
CORS	fruitcake/laravel-cors	^2.0
Excel Export	maatwebsite/excel	^3.1
Activity Logging	spatie/laravel-activitylog	^3.17
Role Management	spatie/laravel-permission	^5.5
Cryptography	OpenSSL (PHP Extension)	-
Server	Apache (.htaccess)	-

3. Key Components

3.1 Mobile App Modules

Module	Files	Responsibility
Authentication	`auth_controller.dart`, `company_auth_screen.dart`, `branch_setup_screen.dart`, `setup_terminal_screen.dart`	Company registration/login, branch creation/login, MoR authentication
Invoice Management	`invoice_controller.dart`, `invoice_generator_screen.dart`, `invoice_detail_screen.dart`	Invoice creation, tax calculation, submission, PDF generation



Module	Files	Responsibility
Invoice History	`invoice_history_controller.dart`, `invoice_history_screen.dart`	Browsing and filtering past invoices
Receipt Management	`receipt_controller.dart`	Sales and withholding receipt submission
Product Catalog	`supplies_screen.dart`	CRUD operations on product catalog items
Offline Queue	`offline_queue_service.dart`	Queue management, connectivity monitoring, auto-sync
Networking	`dio_service.dart`, `dio_config.dart`, `api_service.dart`	HTTP communication, interceptors, retry logic, error handling
Configuration	`config_preference.dart`, `app_settings.dart`	Secure credential storage, app settings
PDF Services	`invoice_pdf_service.dart`, `receipt_pdf_service.dart`	PDF document generation
Theme	`app_theme.dart`, `theme_service.dart`	Light/Dark theme definitions
Localization	`app_translations.dart`	English/Amharic string translations
Navigation	`dashboard_screen.dart`, `initial_navigation_middleware.dart`	Bottom tab navigation, initial route resolution

3.2 Backend Modules

Module	Directory	Responsibility
Company Auth	`Controllers/Api/AuthController.php`	Company registration, login, company listing
Branch Management	`Controllers/Api/BranchController.php`	Branch creation with cert upload, branch login
MoR Gateway	`Controllers/Authentication/`, `Controllers/Registration/`, `Controllers/Cancellation/`, `Controllers/Verification/`, `Controllers/Receipt/`	Proxying signed requests to MoR API endpoints
MoR Services	`Services/`	Business logic for signing, sending, and auditing MoR requests
Signing Trait	`Services/Traits/MorApiServiceTrait.php`	OpenSSL payload signing with SHA-512, certificate encoding
Branch Resolution	`Middleware/ResolveMorBranch.php`	JWT authentication + Branch-Id header verification
Data Models	`Models/`	Eloquent models for all entities
Invoice Storage	`Controllers/Api/MorInvoiceController.php`	Invoice read endpoints (index, show)
Supply Management	`Controllers/SupplyController.php`	Product catalog CRUD

4. Architecture Diagram





```

|
| shared_
| preferences
| (Offline Queue)
|
| cookie_jar
| (HTTP Cookies)
+-----+
|
| mor_invoices
| mor_invoice_audits
| mor_login_audits
| mor_receipts
| mor_cancellation_audits
| supplies
|
| File Storage:
| mor-credentials/
|   {company_id}/
|   {branch_slug}/
|   private_key.key
|   certificate.pem
+-----+
  
```

5. Data Flow Diagram (DFD)

5.1 Level 0 - Context Diagram

```

+-----+
Invoice/Receipt | Signed Fiscal
Requests -----?| Deresegn |---- Documents ----? MoR Gateway
| System |
Auth Credentials ?| |?-- IRN, QR Codes
+-----+
?
|
User Input      Stored Data
(Cashier/Owner) (MySQL + Files)
  
```

5.2 Level 1 - Primary Data Flows

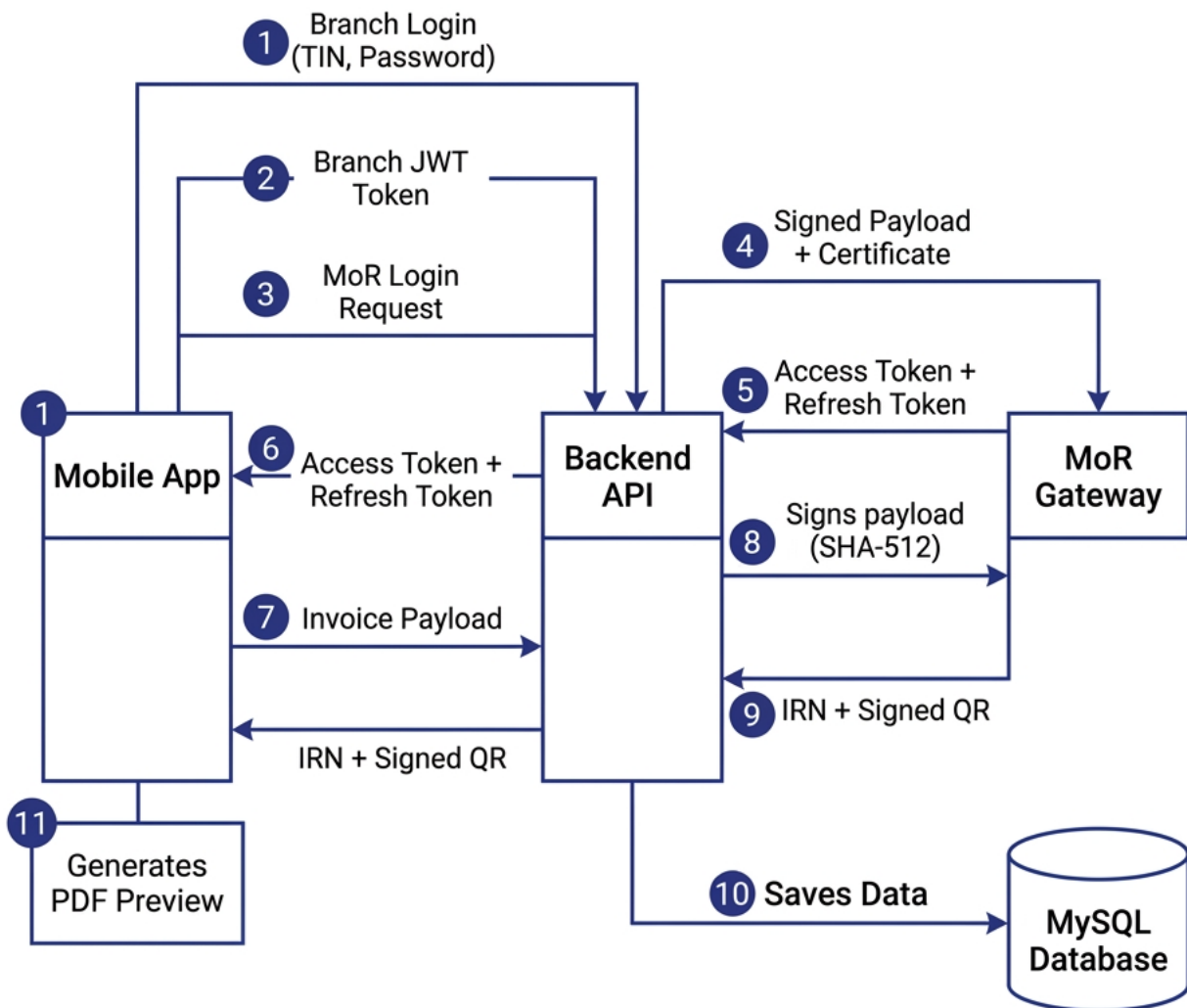


Figure: Data Flow Diagram

6. Data Flow Explanation



6.1 Authentication Flow

- Branch Login: The mobile app sends TIN number and password to `POST /api/branch/login`. The backend validates credentials against the `company\_branches` table (passwords hashed with bcrypt). On success, a JWT is minted for the branch entity using `tymon/jwt-auth`.
- MoR Login: The app then calls `POST /api/login`. The `ResolveMorBranch` middleware authenticates the request via the branch JWT, resolves the branch entity, and passes it to the `LoginController`. The `MorLoginService` loads the branch's private key from disk, signs the login payload with SHA-512, attaches the base64-encoded certificate, and forwards the request to the MoR login URL. The MoR returns an `accessToken` and `refreshToken`, which are relayed back to the mobile app.
- Token Storage: The mobile app stores the branch JWT in `flutter\_secure\_storage` (Android Keystore / iOS Keychain encrypted). MoR tokens are also stored securely. Subsequent API requests include both the branch JWT (`Authorization: Bearer ...`) and the MoR token (`Mor-Token: Bearer ...`) as headers.

6.2 Invoice Registration Flow

- The mobile app constructs a complete invoice payload (document details, seller/buyer, line items, values, payment).
- The payload is sent to `POST /api/invoice/register` with both branch JWT and MoR token headers.
- The `ResolveMorBranch` middleware authenticates and resolves the branch.
- The `InvoiceRegistrationController` invokes `MorRegisterService`, which:
 - Loads the branch's private key and certificate from the filesystem.
 - Serializes the request payload to JSON.
 - Signs the JSON with `openssl\_sign()` using `OPENSSL\_ALGO\_SHA512`.
 - Constructs the outbound payload: `{ request, signature, certificate }`.
 - Sends the signed payload to the MoR registration URL via HTTPS.
- On success, the MoR response (containing IRN, signed QR, acknowledgement date) is:
 - Saved to the `mor\_invoices` table with full buyer/seller/value details.
 - Audited in the `mor\_invoice\_audits` table.
 - Returned to the mobile app.
- The mobile app then automatically registers a sales receipt and generates a combined PDF.

6.3 Offline Synchronization Flow

- When the mobile app detects a network failure during invoice registration (timeout, connection error, `SocketException`), the `OfflineQueueService` saves the invoice payload to `shared\_preferences` as a JSON string.
- The service listens to `Connectivity().onConnectivityChanged` for network state changes.
- When Wi-Fi or mobile data connectivity is restored, `syncQueue()` iterates over queued invoices and attempts to register each one.
- Successfully registered invoices are removed from the queue.



7. Third-Party Services

Service	Purpose	Integration Method
Ethiopian MoR e-Invoicing API	Invoice registration, cancellation, verification, receipt submission	REST API with digitally signed payloads
MySQL Database	Persistent data storage	Eloquent ORM via Laravel
OpenSSL	Digital signature generation (SHA-512) and certificate management	PHP `openssl_sign()` / `openssl_pkey_get_private()`

8. Security Architecture

8.1 Transport Security

- All client-server communication uses HTTPS (TLS 1.2+).
- The backend API is hosted at `https://api.deresegn.com`.
- Dio HTTP client is configured with 60-second connect and 120-second receive timeouts.

8.2 Authentication Security

- Company-level: JWT tokens issued by `tymon/jwt-auth`, validated by the `auth:api` guard.
- Branch-level: Separate JWT guard (`branch`) for branch entities. The `ResolveMorBranch` middleware cross-validates the JWT identity against the `Branch-Id` header.
- MoR-level: Bearer tokens passed through to the MoR gateway; refreshed automatically on 401 responses.

8.3 Data Protection

- Passwords: bcrypt-hashed via Laravel's `Hash::make()`.
- `client\_secret` and `api\_key`: Encrypted at rest using Laravel's `encrypted` Eloquent cast (AES-256-CBC with APP_KEY).
- Private keys: Stored on the filesystem under `storage/app/mor-credentials/` with Unix permissions `0600`.
- Mobile credentials: Stored in `flutter\_secure\_storage` (AES-256 backed by Android Keystore / iOS Keychain).

8.4 Audit Trail

- All MoR login attempts are recorded in `mor\_login\_audits` (sensitive data like `clientSecret` is removed, tokens are masked).
- All invoice registrations are recorded in `mor\_invoice\_audits` with full request/response payloads.
- Invoice cancellations are recorded in `mor\_cancellation\_audits`.

8.5 Sensitive Data Masking

- The `MorLoginService` explicitly unsets `clientSecret` from audit payloads and replaces `accessToken` and `refreshToken` with `MASKED` before persisting.



9. Distribution Model

Platform	Format	Distribution
Android	APK (release signed)	Direct distribution / internal deployment
iOS	IPA	TestFlight / direct distribution

The application is currently distributed as direct APK files for Android devices. It is not listed on the Google Play Store or Apple App Store.

10. Offline/Online Behavior

10.1 Online Mode

- Full functionality: invoice registration, receipt submission, history browsing, product catalog management.
- Real-time MoR API interaction.

10.2 Offline Mode

- Invoice creation is queued locally using `shared\_preferences`.
- Queued invoices are displayed to the user with a pending status.
- Connectivity monitoring via `connectivity\_plus` triggers automatic sync.
- DioService implements retry logic for retryable errors (HTTP 520, `retryable: true` flag) with exponential backoff (capped at 5 seconds).

10.3 Caching

- Product catalog data is fetched on controller initialization and held in reactive state (`RxList`).
- Invoice history supports pagination and is refreshable.
- No aggressive local caching is implemented beyond the offline queue; data freshness is maintained via API calls.